

Approach Summary

Guoyi Chen

1 Introduction

Given that this optimization problem is NP-hard, we decompose the problem into multiple parameter layers and propose corresponding optimizations for each layer to achieve higher efficiency. We categorize parameters into four layers: subgraphs, tensors_to_retain, granularities and traversal_orders. Then, we finalize subgraphs, tensors_to_retain, granularities and traversal_orders one by one. Each layer can be efficiently finalized based on preceding ones.

Since our implementation organizes scheduling logic into modular components where high-level structures are built upon fundamental building blocks, we present our framework bottom-up: from low-level traversal_orders to high-level subgraphs. Specifically, we detail: using the latency feature to build the evaluator, exploring traversal_orders via systematic search, accelerating the search for granularities through decision monotonicity, applying dynamic programming for tensors_to_retain, and finally utilizing simulated annealing to determine the subgraphs.

2 Methods

We sequentially determine the parameters in the following order: subgraphs, tensors_to_retain, granularities, and traversal_orders.

Since other parameter layers are attributes of the subgraphs, it is evident that the optimal parameters for subsequent layers cannot be determined until the subgraph structures are finalized. Because tensors_to_retain influences the remaining memory capacity, which is a crucial factor in the exhaustive search of granularities, we subsequently fix tensors_to_retain after we get a graph structure. Then, given that the number of values in traversal_orders is affected by granularity, we finalize granularity before traversal_orders.

2.1 Latency

After finalizing subgraphs, tensors_to_retain, granularities and traversal_orders, we can efficiently calculate the latency of the subgraph. Given the locality of every subgraph, we can calculate the latency of each subgraph independently. Since the granularity is fixed, we can work backward to determine the required size of the inputs.

Then, because the size of inputs remains constant across iterations and there are only two implicit reuse conditions—row-major or column-major continuity—we can directly obtain the latency in $O(1)$ time through mathematical derivation. While certain technical nuances exist within the mathematical derivation, they follow a straightforward logic and are omitted for brevity.

In total, the latency of each subgraph can be determined with a complexity of $O(\text{size of subgraph})$.

2.2 traversal_orders

Since the traversal orders are only reflected in the coefficients of the mathematical expression, we can employ a greedy algorithm. This

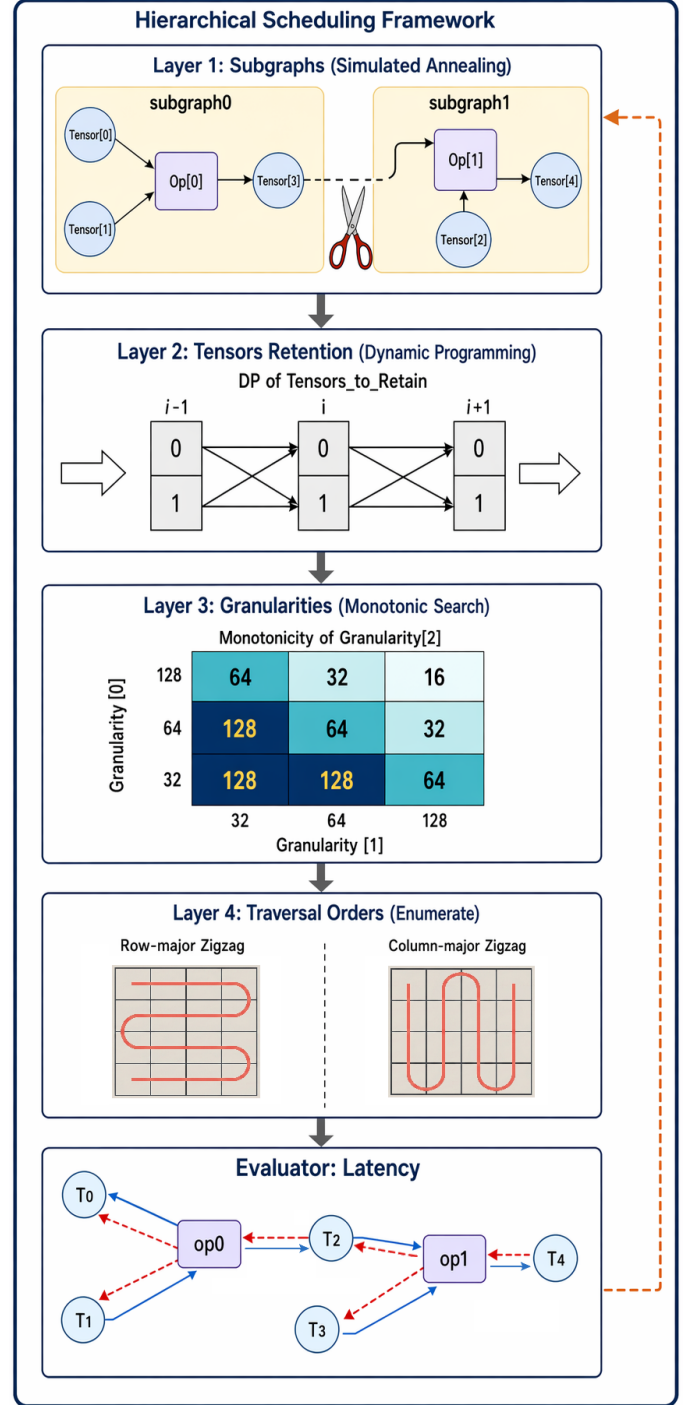


Figure 1: Methodology Framework

implies that there are only two potentially optimal strategies: row-major zigzagging and column-major zigzagging. We can simply enumerate two strategies and return the better one.

2.3 granularities

By leveraging the properties of the latency model, we can adopt a more efficient search strategy for tile dimensions. Specifically, regarding granularity, a greedy approach is feasible where we maximize the value of `granularity[2]` until hardware or memory constraints are exceeded.

Furthermore, since `granularity[2]` is monotonically non-increasing with respect to `granularity[0]` or `granularity[1]`, we can utilize decision monotonicity. Instead of restarting the search for each state, the algorithm inherits the `granularity[2]` value from the previous iteration and performs a downward search until a valid configuration is determined. Finally, we calculate the latency of each feasible granularity and select the optimal one.

If we assume the number of possible solutions for `granularity[0]`, `granularity[1]`, and `granularity[2]` is on the same scale, denoted as $O(cnt)$, our complexity analysis shows that the algorithm performs both memory legitimacy checks and latency calculations, with the total number of operations bounded by $O(cnt^2)$.

2.4 tensors_to_retain

Each retained tensor is stored in fast memory only for the duration of the current and subsequent subgraphs. This locality of the `tensors_to_retain` effect is a favorable property for Dynamic Programming.

We define the DP state as $f[i][0/1]$, where i represents the subgraph currently being processed, and the boolean value indicates whether its output is retained. To simplify the problem, we have imposed a constraint that each subgraph has exactly one output. When transitioning from $f[i-1][k]$ to $f[i][j]$, we enumerate all combinations of whether the previous and current outputs are retained ($j, k \in \{0, 1\}$), and then select the optimal strategy.

Once the local `tensors_to_retain` for the current subgraph are determined, we calculate the latency using the aforementioned method. Specifically, the transition follows the recurrence:

$$f[i][j] = \min(f[i][j], f[i-1][k] + \text{calculate_latency}(i, j, k))$$

The primary advantage of this approach is that we only need to compute the latency for the current subgraph instead of the total graph during each step. In conclusion, our algorithm not only determines the `tensors_to_retain`, granularities, and `traversal_orders` within a complexity of $O((n+m) \cdot cnt^2)$, but also guarantees a globally optimal solution under the specific subgraph configurations.

2.5 subgraphs

Because the subgraphs are the first to be fixed, there is minimal prior information available to guide our determination. Consequently, we choose simulated annealing as the core methods for this layer, randomly fusing operators to fix the subgraphs.

Every tensor possesses a boolean attribute `is_cut`. If `is_cut` is set to true, the edges between that tensor and its consumer nodes

are cut. By regarding the remaining edges as undirected, we can fuse each connected part into a single subgraph and order every part via topological sort. Finally, we get a version of subgraphs. Specifically, there are some invalid conditions that need to be excluded, such as the dependency cycles.

The simulated annealing process starts with a totally partitioned graph, in which the `is_cut` attribute of every tensor is set to true. In each iteration, we randomly select a tensor and flip its `is_cut` attribute. The efficacy of this approach is guaranteed because any subgraph configuration can be achieved by toggling edges individually, ensuring that every possible state is reachable from the initial state.

Because the total latency of different strategies varies significantly, we set the initial temperature to ten times the initial latency. Additionally, we use a terminal temperature of 0.01 and a cooling rate of 0.995.

3 Results

Ultimately, the code achieved the theoretical optimal solution on five examples and yielded performance improvements across five benchmarks. The final results are as follows:

Case	Total_Latency
Example 1	3276.800000
Example 2	13107.200000
Example 3	4500.000000
Example 4	4915.200000
Example 5	6734.400000
mlsys-2026-1	471500.800000
mlsys-2026-5	885828.266667
mlsys-2026-9	167530987.520000
mlsys-2026-13	166405242.880000
mlsys-2026-17	46326400.000000

4 Discussion

While our proposed approach achieves promising results, there remains room for further improvement.

The use of simulated annealing introduces the risk of converging to local minima during the search process. This issue could potentially be mitigated by replacing it with a tree-based heuristic search.

To achieve a simpler and more unambiguous implementation, we introduce several simplifications to the original problem, such as restricting each subgraph to a single output. By progressively addressing these simplified instances, the overall performance may be further improved.

Based on this problem formulation, future work could relax some of the simplifications and abstractions to more closely align with actual computational workloads, unlocking its potential for real-world application.